VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BCSE307P – COMPILER DESIGN LAB

PROJECT REPORT

CodeTrace: Compiler-Driven Program and Security Analysis for a C Subset

Submitted by

[Srikiran –24BCE2215]

[Dhruv – 24BCE2211]

[Prince – 24BDS0153]

Under the guidance of

Nithya K,

Assistant Professor Senior Grade II

SEPTEMBER, 2026

BONAFIDE CERTIFICATE

This is to certify that the project work titled CodeTrace: Compiler-Driven Program and Security Analysis for a C Subset is a bonafide record of work carried out by [Team Member Names and Registration Numbers], submitted in partial fulfilment of the requirements for the course BCSE307P – Compiler Design Lab, School of Computer Science and Engineering, VIT Vellore, during the academic year 2026 - 2027.

Faculty Guide: _____

Date: _____

DECLARATION

We hereby declare that the project report titled CodeTrace: Compiler-Driven Program and Security Analysis for a C Subset submitted by us to VIT Vellore in partial fulfilment of the requirement for the award of the degree/course BCSE307P – Compiler Design Lab is a record of bona fide work carried out by us under the supervision of Nithya K. The contents of this report, in full or in part, have not been submitted to any other institute or university for the award of any degree or diploma.

Place: _____

Date: _____

[Signatures of all team members]

ABSTRACT

CodeTrace is a static program-analysis platform for CT-C, a documented subset of C. It combines compiler construction with program-dependency and security analysis in one inspectable workflow. The lexer produces tokens with source locations, and a grammar-driven LR parser builds a syntax tree for semantic analysis. Scoped symbol information is then used to construct per-function control-flow graphs and to compute reaching definitions, liveness and definite-assignment information. A declarative rule language specifies taint sources, sinks, sanitizers and validators. The analysis follows explicit data flow through assignments, expressions, function arguments and return values, then relates the result to a typed dependency graph and forward, backward or finding-focused program slices. Findings include a source-to-sink explanation, certainty, limitations and remediation guidance. A browser interface and HTTP API expose these artefacts, and JSON and self-contained HTML reports support review outside the interactive interface. CodeTrace does not execute submitted programs. Its supported language is deliberately limited, and reported findings describe potential flows rather than proven exploits. The project's central contribution is educational: compiler representations and analysis reasoning are visible alongside the security result, making the process easier to inspect and explain.

TABLE OF CONTENTS

ABSTRACT 4
CHAPTER 1 INTRODUCTION 6
1.1 Background 6
1.2 Problem Statement 6
1.3 Objectives & Novelty 7
CHAPTER 2 LITERATURE SURVEY 8
2.1 Existing Systems / Related Work 8
2.2 Research Gap 9
CHAPTER 3 SYSTEM DESIGN 10
3.1 Proposed System Architecture 10
3.2 Module Description 11
3.3 Compiler Phases Implementation 11
CHAPTER 4 IMPLEMENTATION DETAILS 12
4.1 Technology and Application Workflow 12
4.2 Worked Example: Command Injection 12
Application Screenshots and Output Evidence 13
Current Progress and Challenges 13
Conclusion and Future Enhancements 13
REFERENCES 14

CHAPTER 1

INTRODUCTION

1.1 Background:

A compiler front end converts source text into a structured representation that later stages can reason about. Lexical analysis groups characters into tokens, syntax analysis checks whether the token sequence follows a grammar, and semantic analysis resolves declarations and checks relationships between names and operations. These steps are useful beyond translation: once a program is represented as an abstract syntax tree and its identifiers are resolved, an analysis can reason about how values move through statements and functions [1].

Static program analysis derives properties of a program without running the submitted program. Data-flow analysis evaluates facts at program points by following control flow, while taint analysis follows values from designated sources to operations that require particular handling. These methods are relevant to injection risks, where externally influenced data may reach an operation such as a command interpreter, database query, file path, or format-string function. The analysis must distinguish a call that merely exists from a call whose argument may be influenced by untrusted input [4, 5, 9].

CodeTrace applies these ideas to CT-C, a documented, deliberately small C-like language subset. The language front end, intermediate representations and analysis results are made available for inspection through a browser interface and an HTTP API. The aim is to show how compiler-design components support a traceable security analysis, rather than to provide a full C compiler or to claim comprehensive vulnerability detection.

1.2 Problem Statement:

Security findings are difficult to evaluate when a tool reports only a rule name and a source line. Without a model of syntax, symbol identity and control flow, it is difficult to explain whether a value can reach a sensitive operation, which assignments and function calls connect the two, or where an analysis assumption affects the conclusion. A second problem is interpreting an empty result: the absence of a reported flow is meaningful only in relation to the language constructs, rules and analysis approximations actually considered.

The project addresses the following problem: given a single source file written in the supported CT-C subset, construct the compiler representations needed for analysis, derive selected control-flow and data-flow facts, identify potential rule-defined source-to-sink flows, and present the reasoning and limits behind each finding. The analysed source must remain data; it is not executed as part of the analysis.

1.3 Objectives & Novelty:

The main objectives are to:

- Implement a lexer that preserves source positions and provides located diagnostics for malformed and unsupported input.
- Generate LR-family parsing tables from the CT-C grammar and build a source-located abstract syntax tree.
- Resolve declarations with scoped symbol tables and report semantic issues that affect the reliability of later analysis.
- Construct per-function control-flow graphs and compute reaching definitions, liveness and definite assignment using a shared worklist framework.
- Specify taint sources, sinks, sanitizers, validators and flows in a declarative rule language, then track explicit data flow across statements and supported function calls.
- Build typed dependency relationships and forward, backward and finding-focused slices to explain the statements related to a result.
- Present findings through a browser and HTTP API, including an evidence path, certainty, known limitations and remediation guidance.

Contribution and scope:

The contribution is the integration of these established methods into a small, inspectable pipeline. A shared grammar-processing approach is used for CT-C and the rule language, and later stages work from parsed structures and resolved symbols rather than searching raw source text for names. Findings are accompanied by the path and assumptions used to justify them, so the reader can connect the security result to the compiler artefacts that support it.

The scope is intentionally bounded. CT-C supports selected scalar types, fixed-size array declarations, expressions, functions, conditionals and loops. Full C features such as pointer and alias analysis, preprocessor semantics and multi-file linking are outside the implemented language model. The resulting findings should therefore be read as potential flows within the supported subset and configured rules, not as proof of exploitability or a guarantee that a program is free of vulnerabilities.

CHAPTER 2

LITERATURE SURVEY

2.1 Existing Systems / Related Work:

Compiler construction provides the front-end methods used by CodeTrace. Standard compiler texts describe the progression from tokens to syntax trees and semantic representations [1]. Knuth established the LR approach to parsing from left to right [2], while DeRemer's simple LR method provides a compact table-construction approach [3]. CodeTrace applies an LR-family, table-driven parser to its own defined grammar; it does not rely on a general-purpose C compiler to produce the structures used by the analysis.

Data-flow analysis provides a formal way to compute program facts across a control-flow graph. Kildall's framework unified several global program analyses using transfer functions and iteration [4], while Nielson, Nielson and Hankin give a systematic treatment of program analysis and its underlying lattices and fixed-point methods [5]. CodeTrace applies this general approach to reaching definitions, liveness and definite assignment. The results help explain which definitions may reach a statement and which values may be used later.

Program slicing addresses program comprehension by selecting statements relevant to a chosen value or computation. Weiser's work introduced program slicing as a way to isolate statements related to a slicing criterion [6]. Dependence-graph methods model data and control relationships explicitly [7]. Interprocedural analysis extends reasoning across function boundaries; Reps, Horwitz and Sagiv formulate a graph-reachability approach for precise interprocedural data-flow problems under defined assumptions [8]. CodeTrace uses a typed dependency graph to make selected relationships visible and computes slices around statements or findings.

Security-oriented static analysis uses program structure and rules to identify potential weaknesses without executing the target program. Static analysis can trace flows that simple function-name checks cannot, but it remains dependent on the language model, source and sink definitions, handling rules and analysis approximations [9, 10]. CodeTrace adopts this source-to-sink model for a small language subset and makes the evidence path and uncertainty part of the reported result.

Approach	Typical strength	Relevance to CodeTrace
Compiler construction	Formal recognition of syntax and production of structured program representations.	Supplies the lexer, LR parser, AST and semantic foundation for analysis.
Data-flow frameworks	Systematic computation of facts over control-flow graphs.	Supports reaching definitions, liveness and definite assignment.
Program slicing and dependence graphs	Identifies statements related through data or control dependencies.	Provides explanations and finding-focused views of connected statements.
Security-oriented static analysis	Checks whether data from configured sources may reach sensitive operations.	Forms the basis of rule-driven taint findings and remediation guidance.

2.2 Research Gap:

The gap addressed by CodeTrace is primarily one of integration and inspectability in a Compiler Design learning context. Textbooks explain individual algorithms, and security analysers may present their conclusions through findings and rule identifiers, but a learner often needs to connect several stages to understand why a result appears. CodeTrace places the lexer and parser, syntax tree, scoped symbols, control-flow graph, data-flow facts, taint propagation and dependency relationships in a single workflow over the same source program.

A second design concern is the interpretation of negative results. A list with no findings does not establish that the source is safe. The tool therefore records which constructs are unsupported, marks an analysis as partial when diagnostics or limits affect the result, and exposes the reasons why inspected sink calls were not reported where that information is available. This makes the boundary of the analysis visible instead of allowing a quiet result to be mistaken for a complete proof.

A third concern is explanation. A useful security finding should state the source and sink, show the sequence of data-flow steps, identify relevant sanitizers or validator assumptions, describe its certainty, and state limitations that affect its interpretation. The typed dependency graph and program slice support a second view of the same result: the evidence path describes the flow, while the graph and slice show related statements and relationships.

Positioning of the proposed system:

CodeTrace is not intended to replace mature static application security testing tools or to claim greater detection accuracy than them. Its contribution is that the underlying compiler and analysis representations are available for inspection, and that the implementation runs without third-party runtime packages or a separate build step. This makes the relationship between compiler construction, data-flow analysis and security reasoning directly observable in an academic project.

The research gap is therefore addressed within a constrained language, and the same constraint is a limitation. The system does not model all constructs found in production C, and its interprocedural summaries and array treatment remain approximate. These boundaries are stated here because they affect what can be concluded from the analysis and define the most relevant directions for future development.

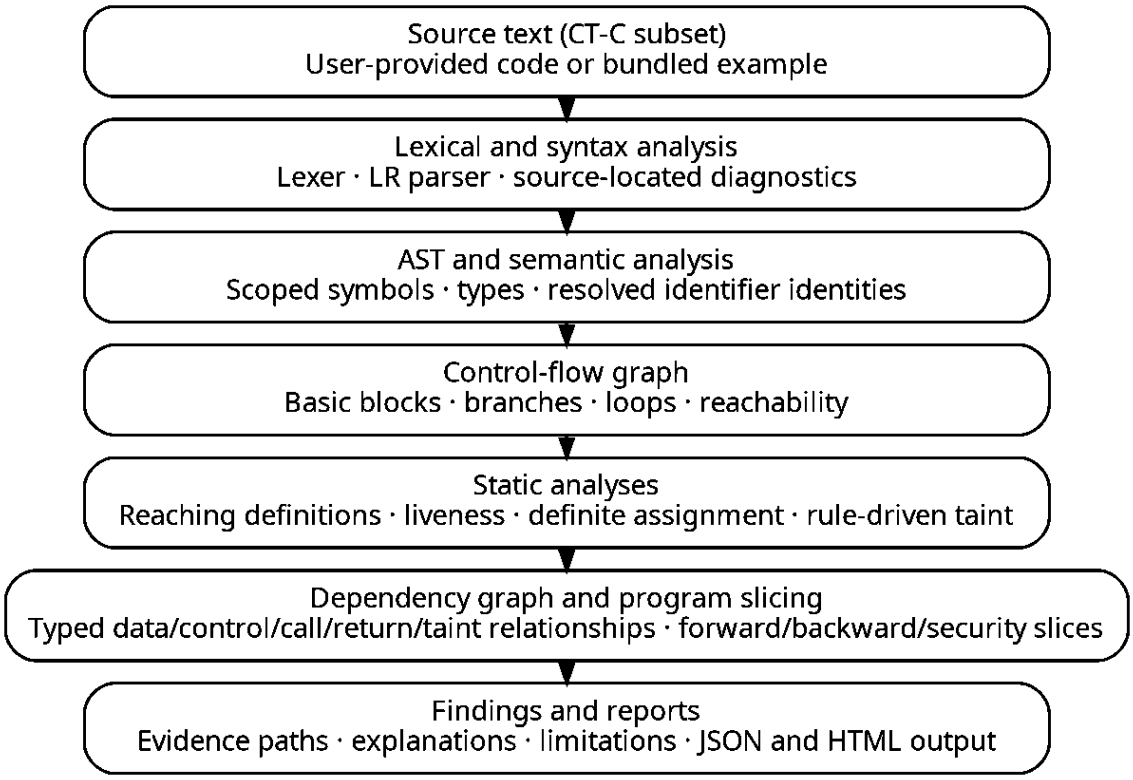
Design concern	CodeTrace response	Boundary
Inspectability	Expose token, syntax, graph, data-flow and evidence views.	Views reflect only the implemented model.
Negative results	Show unsupported input, partial-analysis reasons and unreported-sink dispositions where available.	No finding is not a proof of safety.
Explainability	Attach source/sink details, flow steps, certainty and remediation.	The path may be approximate.
Learning value	Use one source representation across compiler and analysis stages.	The supported language is intentionally small.

CHAPTER 3

SYSTEM DESIGN

3.1 Proposed System Architecture:

CodeTrace separates the browser interface from the analysis engine. The browser submits source text and options to the Python HTTP API. The API validates the request and invokes the analysis pipeline; the engine returns a program object containing the intermediate representations, diagnostics, data-flow results, taint facts, dependency graph, slices and findings. The API exposes those results to the interface and to the JSON and HTML reporting modules.



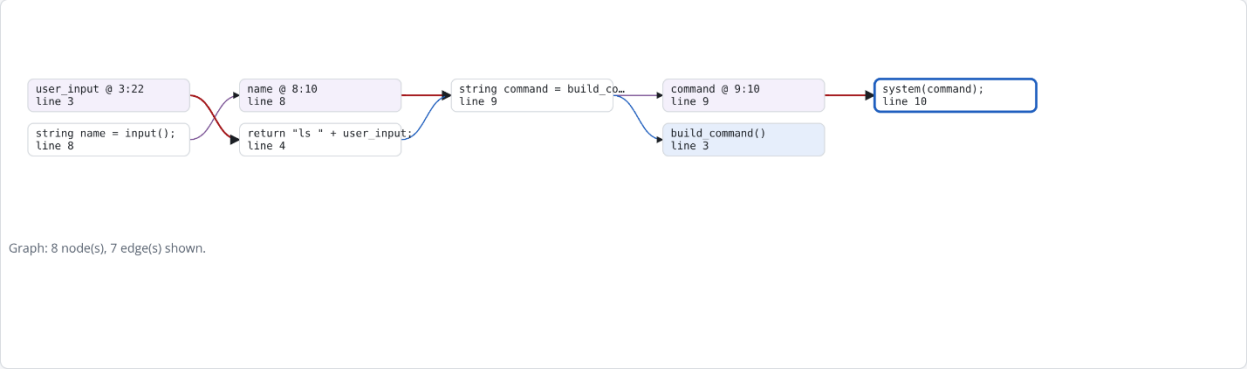
The analysis pipeline proceeds through lexical analysis, parsing, semantic analysis and construction of a control-flow graph. Data-flow analysis supplies facts used by taint propagation. A typed dependency graph is then built to represent data, control, call, return and taint relationships. Slicing and finding explanation consume these relationships before the reporting stage produces a stable, reviewable result.

The browser is a presentation layer rather than a second analysis engine. It displays the structured output received from the server, supports navigation between analysis views, and requests detail for a run. The server keeps recent runs in memory; it does not use a database. CodeTrace is designed for local use, and the analysed program is treated as input data rather than executed code.

3.2 Module Description:

The implementation is divided into packages with defined responsibilities. These boundaries allow each stage to operate on explicit data structures and give the interface a consistent representation of an analysis run.

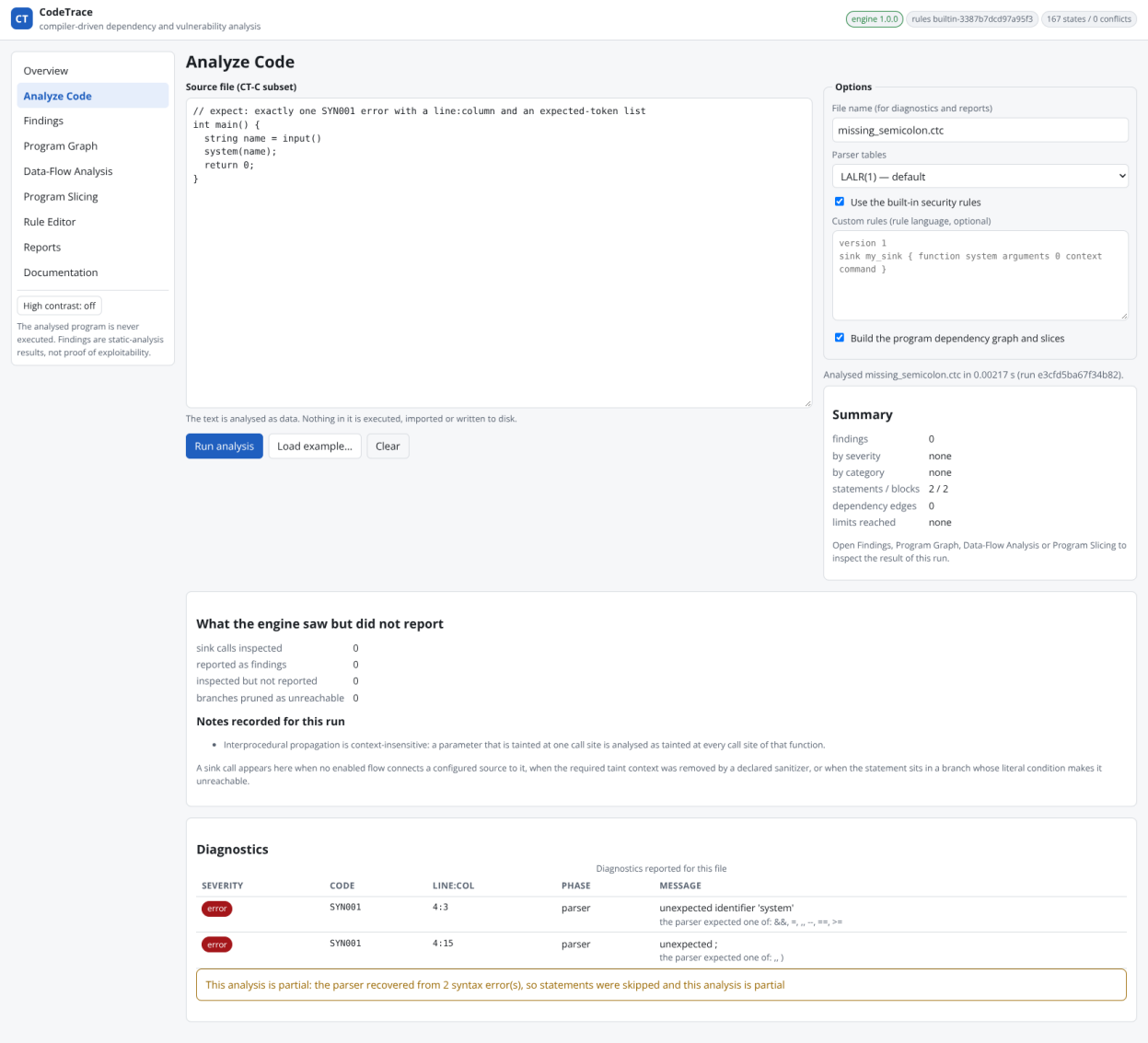
Module	Responsibility	Main output
Language front end	Tokenization, grammar analysis, LR-family tables, parser and AST node definitions.	Located tokens, syntax tree and syntax diagnostics.
Semantic analysis	Scope handling, stable symbol identities, type and declaration checks, and per-statement reads/writes.	Resolved symbols and semantic diagnostics.
Control-flow and data flow	Per-function basic blocks, typed edges, reachability and worklist analyses.	CFGs, definitions, reaching-definition, liveness and definite-assignment states.
Rules and taint analysis	Declarative rule parsing; source, sink, sanitizer, validator and flow matching.	Taint paths, summaries, dispositions and certainty information.
Dependencies and slicing	Typed dependence edges and statement reachability for a selected criterion.	Program dependency graph and forward, backward or finding-focused slices.
Findings, reporting and API	Explanations, stable finding identity, JSON/HTML serialization, HTTP routing and request validation.	Interactive results and exported reports.



3.3 Compiler Phases Implementation:

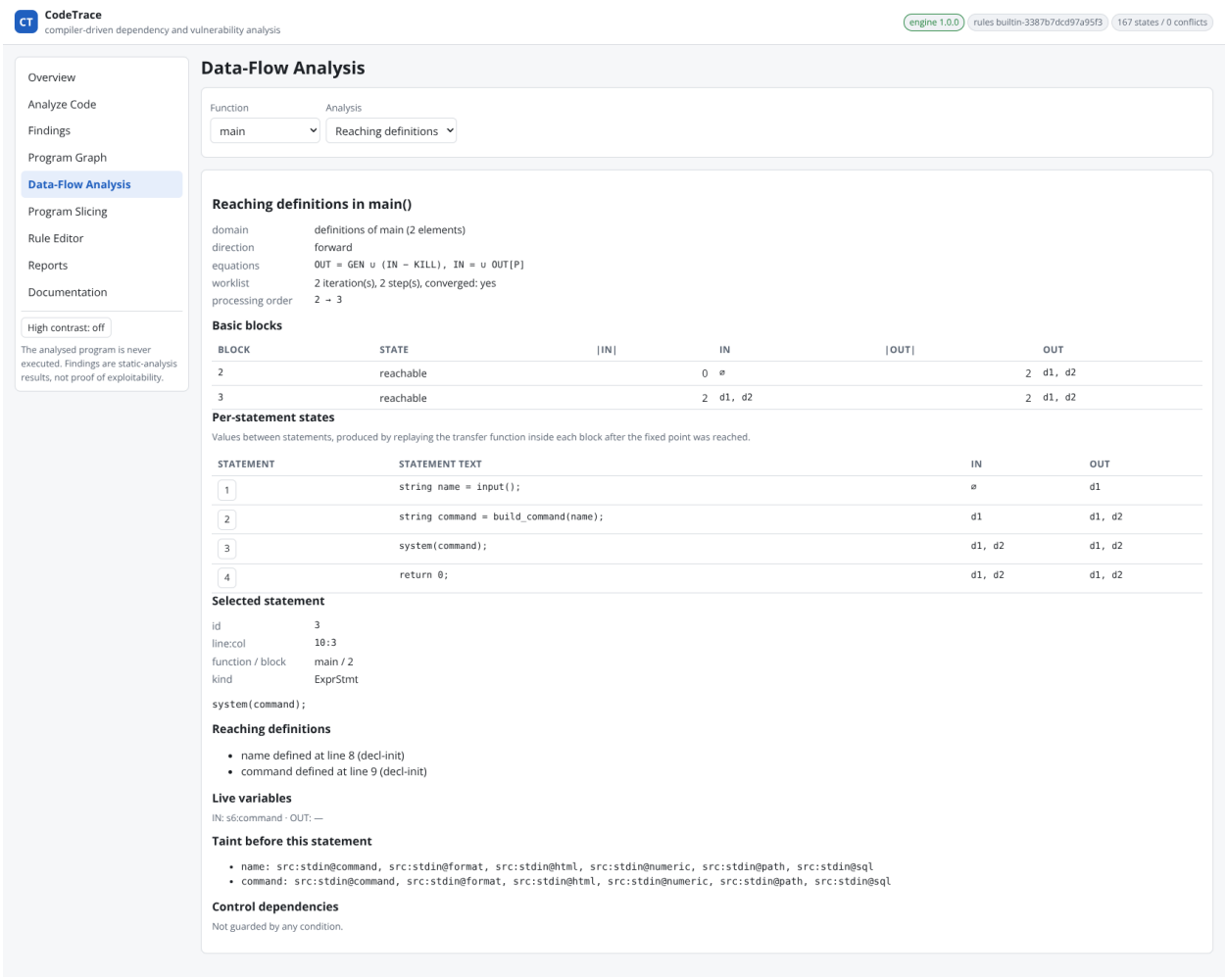
Lexical analysis. The lexer converts source characters into typed tokens with offsets and line-and-column positions. It skips comments and recognises the supported identifiers, literals, punctuation and operators. Malformed literals and constructs outside the CT-C subset receive explicit diagnostics.

Syntax analysis and AST construction. A grammar-driven LR-family parser recognises supported declarations, statements and expressions. Parser modes expose canonical LR(1), LALR(1) and SLR(1) table construction. The parser produces located syntax diagnostics and recovery information; accepted input is represented as an abstract syntax tree whose nodes carry source spans.



Semantic analysis and intermediate representation. Nested symbol tables distinguish declarations with the same spelling in different scopes. Semantic checks identify issues such as unresolved names, incompatible operations and incorrect call arguments. The AST is then converted into per-function control-flow graphs whose basic blocks and typed edges model sequencing, branches, loops and returns.

Data-flow, taint and dependency analysis. A shared worklist framework computes reaching definitions, liveness and definite assignment. Rule-driven taint analysis follows explicit data flow through assignments, expressions and supported function calls. The dependency graph records the reasons for data, control, call, return and taint edges; slices use those edges to isolate statements relevant to a chosen statement or finding.


CT
CodeTrace
compiler-driven dependency and vulnerability analysis
engine 1.0.0
rules builtin-3387b7dcd97a95f3
167 states / 0 conflicts
Overview
Analyze Code
Findings
Program Graph
Data-Flow Analysis
Program Slicing
Rule Editor
Reports
Documentation
High contrast: off
The analysed program is never executed. Findings are static-analysis results, not proof of exploitability.
Data-Flow Analysis
Function
main
Analysis
Reaching definitions
Reaching definitions in main()
domain definitions of main (2 elements)
direction forward
equations OUT = GEN ∪ (IN − KILL), IN = ∪ OUT[P]
worklist 2 iteration(s), 2 step(s), converged: yes
processing order 2 → 3
Basic blocks
BLOCK STATE |IN| IN |OUT| OUT
2 reachable 0 ∅ 2 d1, d2
3 reachable 2 d1, d2 2 d1, d2
Per-statement states
Values between statements, produced by replaying the transfer function inside each block after the fixed point was reached.
STATEMENT STATEMENT TEXT IN OUT
1 string name = input(); ∅ d1
2 string command = build_command(name); d1 d1, d2
3 system(command); d1, d2 d1, d2
4 return 0; d1, d2 d1, d2
Selected statement
id 3
line:col 10:3
function / block main / 2
kind ExprStmt
system(command);
Reaching definitions
name defined at line 8 (decl-init)
command defined at line 9 (decl-init)
Live variables
IN: s6:command · OUT: —
Taint before this statement
name: src:stdin@command, src:stdin@format, src:stdin@html, src:stdin@numeric, src:stdin@path, src:stdin@sql
command: src:stdin@command, src:stdin@format, src:stdin@html, src:stdin@numeric, src:stdin@path, src:stdin@sql
Control dependencies
Not guarded by any condition.

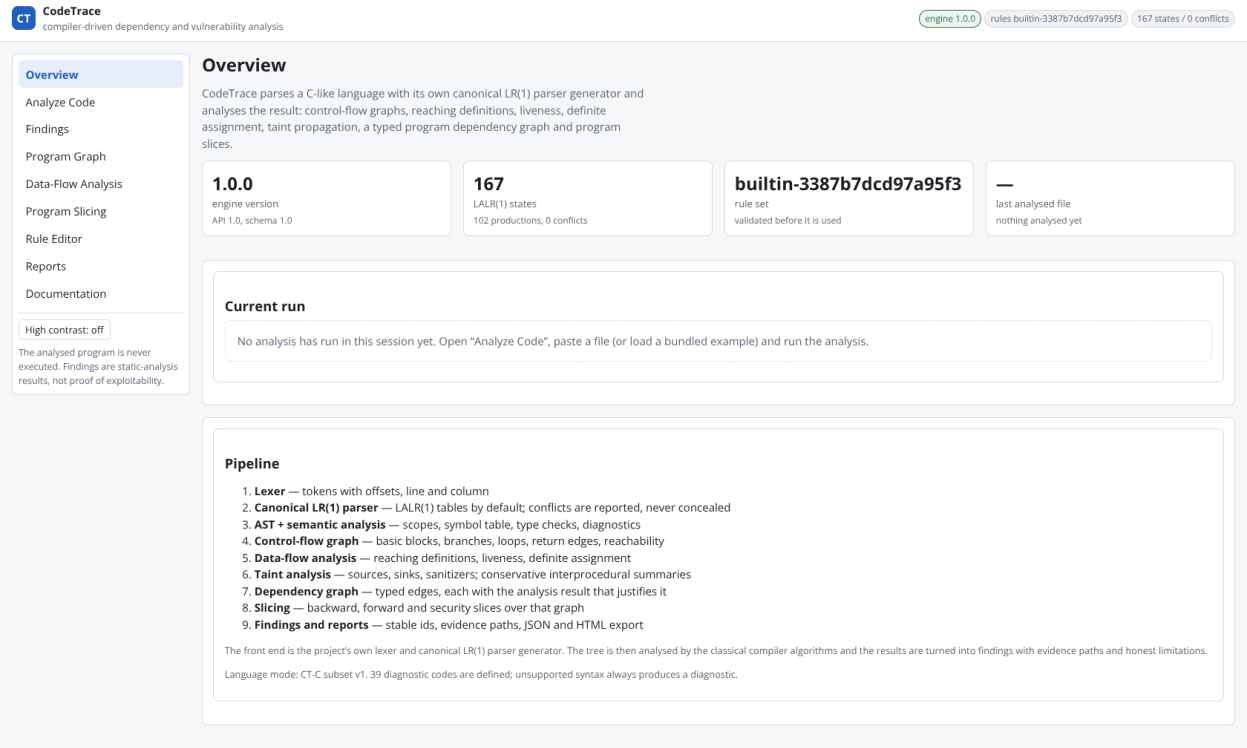
CHAPTER 4

IMPLEMENTATION DETAILS

4.1 Technology and Application Workflow:

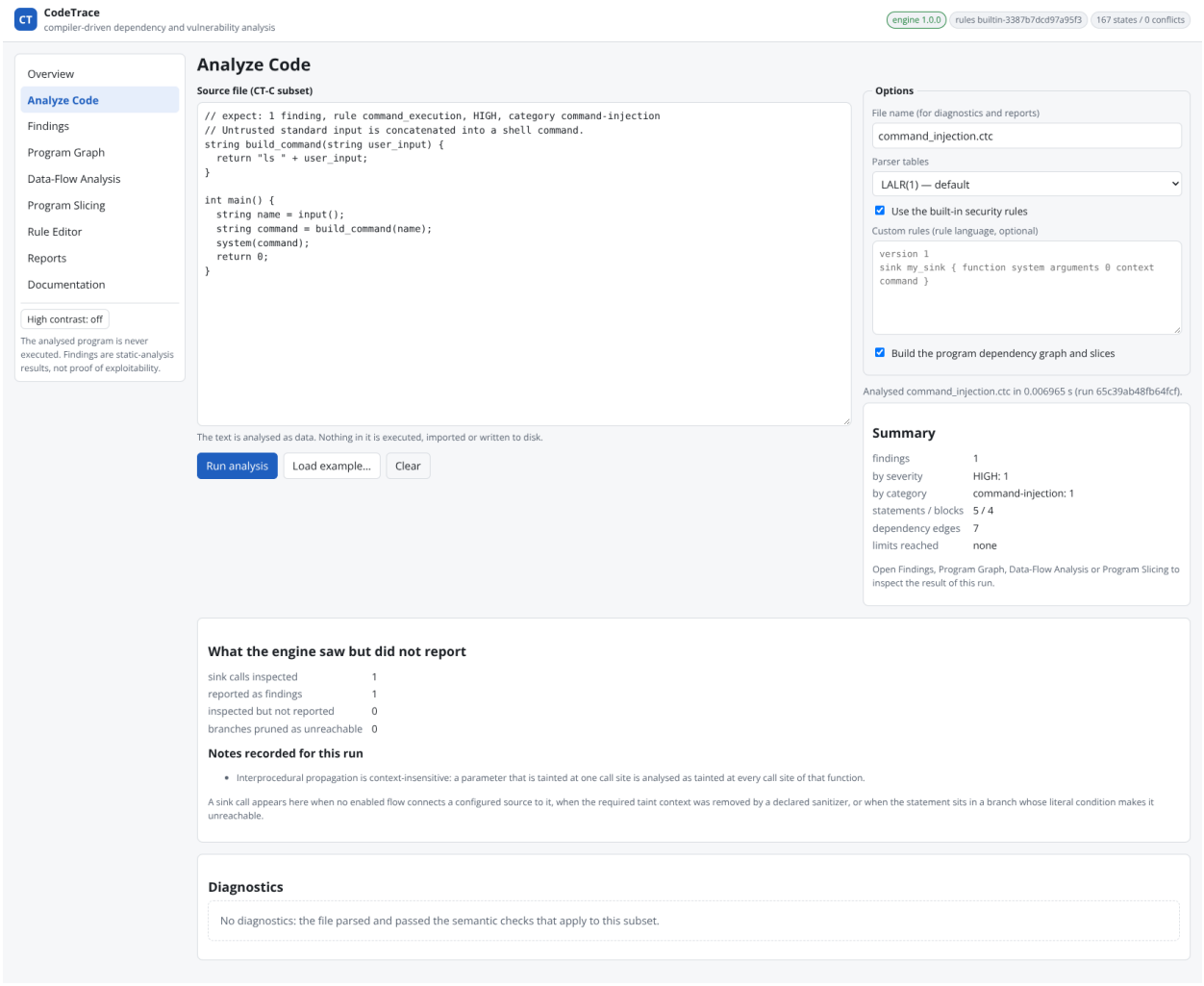
The analysis engine is implemented in Python 3.11 using the standard library. The browser application uses HTML, CSS and plain JavaScript ES modules and has no separate build step or third-party runtime package. A local HTTP server exposes analysis operations and structured result data. The project does not require a database, API key or cloud service for its documented local workflow.

A typical session begins when the user enters CT-C source code or selects a bundled example. The interface submits the source, filename, parser mode and rule options. The server validates the request and invokes the analysis pipeline. The interface then presents diagnostics, a run summary, findings and supporting views such as the program graph, data-flow states and slices. A user can inspect a finding, follow its evidence path and export the result as JSON or HTML.



4.2 Worked Example: Command Injection:

The following excerpt illustrates a command string assembled from untrusted input and passed to a sensitive operation. It is drawn from the bundled command-injection example; comments and unrelated lines are omitted for clarity.




```
string build_command(string user_input) {
    return "ls " + user_input;
}

int main() {
    string name = input();
    string command = build_command(name);
    system(command);
    return 0;
}
```

HIGH command-injection

finding id	`FEF13A28F7E0E`
flow rule	command_execution
confidence / certainty	medium / approximate
reasoning	interprocedural (a function boundary was crossed)
path feasibility	unconditional — no branch condition guards the reported path; the path is structurally reachable from the entry of the enclosing function.
sink	system() line 10:10 in main()
source	stdin (input) line 8

Why this was reported

Potential command injection flow detected: data introduced at line 8 (input) reaches argument 0 of system() at line 10, which rule 'command_system' classifies as a 'command' context operation. The taint engine propagated the atom(s) src:stdin@command along 7 recorded step(s): input() introduces untrusted data (source rule 'stdin') (line 8) -> 'name' is assigned a tainted value at line 8 (line 8) -> argument 0 of the call to build_command() is tainted (line 9) -> 'user_input' is a parameter of build_command() (line 3) -> build_command() returns a tainted value (line 4) -> call to build_command() propagates taint to the caller (line 9) -> 'command' is assigned a tainted value at line 9 (line 9). The flow crosses at least one function boundary, so the interprocedural summary was used. The flow rule 'command_execution' connects source 'stdin' to sink 'command_system' and lists sanitizers shell_escape, parse_to_int, validate_command; none of them removed the atoms that reached this sink. Analysis certainty: approximate. This is a static finding: exploitability was not established.

Evidence path

1. input() introduces untrusted data (source rule 'stdin')
   source · line 8:17 · main()
   one line read from standard input; attacker controlled in most deployments
2. 'name' is assigned a tainted value at line 8
   declaration · line 8:3 · main()
   name declared and initialised
3. argument 0 of the call to build_command() is tainted
   argument · line 9:34 · main()
   the caller passes this value to parameter 0 ('user_input') at line 9
4. 'user_input' is a parameter of build_command()
   parameter · line 3:22 · build_command()
   the value enters this function through its parameter list
5. build_command() returns a tainted value
   return · line 4:3 · build_command()
   the returned expression carries the same taint atom
6. call to build_command() propagates taint to the caller
   call · line 9:20 · main()
   the function summary records that the arguments flowing into this call influence the returned value
7. 'command' is assigned a tainted value at line 9
   declaration · line 9:3 · main()
   command declared and initialised
8. system() argument 0 consumes data that is not safe for context 'command'
   sink · line 10:10 · main()
   sink rule 'command_system' (executes its argument through the system command interpreter) matched by the taint engine

Limitations of this finding

- Interprocedural reasoning is context-insensitive: parameters are summarised across all call sites, which can add flows that a single call site cannot actually create.
- The flow was detected by static analysis only; the analysed program was never executed and exploitability was not established.

Remediation

Do not build a command from untrusted input. Prefer a fixed program path with arguments passed as separate values; if a shell string is unavoidable, escape the value for the command context (for example with the 'shell_escape' rule) and re-run the analysis to confirm the flow disappears.

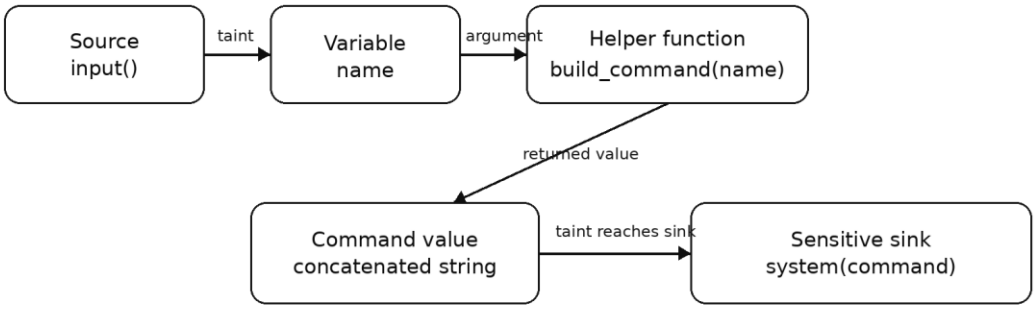
Source context

```
 1  // expect: 1 finding, rule command_execution, HIGH, category command-injection
 2  // Untrusted standard input is concatenated into a shell command.
 3  string build_command(string user_input) {
 4    return "ls " + user_input;
 5  }
 6
 7  int main() {
 8    string name = input();
 9    string command = build_command(name);
10    system(command);
11    return 0;
12  }
```

Show in Program Graph | Security slice of this finding

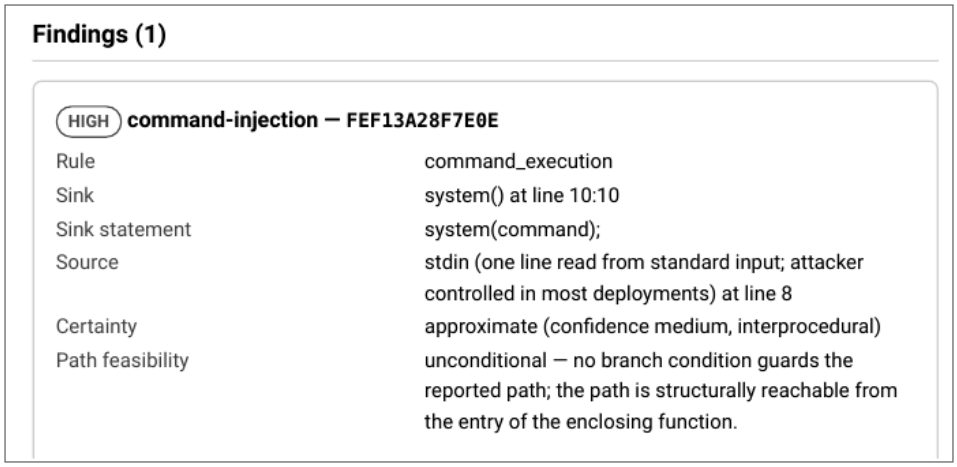
▸ **Evidence record (what justified each hop)**

The configured source rule identifies input() as a source of externally influenced data. The value flows into name, crosses the build_command function boundary, is combined with a command prefix, and reaches argument zero of system(). The analyser reports a potential command-injection flow and supplies the steps used to justify it. Its certainty is approximate because the interprocedural summary is context-insensitive; exploitability is not established. A safer design passes arguments separately to a fixed executable.



Application Screenshots and Output Evidence:

The report excerpt records the source, sink, category and certainty of a potential flow. It is genuine output generated by CodeTrace, not a browser-window screenshot.



Malformed input produces a located diagnostic and a partial-analysis status. Table 3 shows a representative diagnostic from the missing-semicolon example.

Input condition	Diagnostic excerpt	Interpretation
Missing semicolon	SYN001; parser; 4:3 — unexpected identifier "system"; expected-token list provided.	The grammar no longer matches; the recovered run is marked partial.

Current Progress and Challenges:

CodeTrace connects CT-C parsing and semantic checks to control-flow, data-flow, taint, dependency, slicing and reporting modules. A central design challenge is preserving source locations and symbol identities across stages so that each result remains traceable to the relevant statements.

The main limitations are the small language subset and conservative analysis: pointers, alias analysis, preprocessor semantics and multi-file programs are outside scope; arrays are not tracked element-wise; implicit flows are not modelled; and interprocedural summaries are context-insensitive. Validator rules depend on declared assumptions, so findings can remain partial or approximate.

Conclusion and Future Enhancements:

CodeTrace demonstrates how compiler representations support explainable static program and security analysis. It connects grammar-based parsing and semantic analysis to control-flow and data-flow reasoning, then presents dependency and taint relationships through findings, slices and reports. Its contribution is inspectability within a documented language subset, not complete C support or proof of exploitability. Future work should prioritise context-sensitive function summaries, stronger branch reasoning, finer-grained array and alias modelling, broader language coverage, independent external evaluation and additional report formats.

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Boston, MA, USA: Pearson/Addison-Wesley, 2006.

[2] D. E. Knuth, "On the translation of languages from left to right," Information and Control, vol. 8, no. 6, pp. 607–639, 1965.

[3] F. L. DeRemer, "Simple LR(k) grammars," Communications of the ACM, vol. 14, no. 7, pp. 453–460, 1971.

[4] G. A. Kildall, "A unified approach to global program optimization," in Proceedings of the ACM Symposium on Principles of Programming Languages (POPL), 1973, pp. 194–206.

[5] F. Nielson, H. R. Nielson, and C. Hankin, Principles of Program Analysis. Berlin, Germany: Springer, 1999.

[6] M. Weiser, "Program slicing," IEEE Transactions on Software Engineering, vol. SE-10, no. 4, pp. 352–357, 1984.

[7] J. Ferrante, K. J. Ottenstein, and J. D. Warren, "The program dependence graph and its use in optimization," ACM Transactions on Programming Languages and Systems, vol. 9, no. 3, pp. 319–349, 1987.

[8] T. Reps, S. Horwitz, and M. Sagiv, "Precise interprocedural dataflow analysis via graph reachability," in Proceedings of the ACM Symposium on Principles of Programming Languages (POPL), 1995, pp. 49–61.

[9] B. Chess and G. McGraw, "Static analysis for security," IEEE Security & Privacy, vol. 2, no. 6, pp. 76–79, 2004.

[10] MITRE, "Common Weakness Enumeration (CWE)," including CWE-22, CWE-78, CWE-89 and CWE-134. [Online]. Available: https://cwe.mitre.org/.